~ Subiecte grilă ~

- 24 ianuarie 2020

2. SUBIECTUL 2

2.1.
```
void f(char *s) {
    switch (s[0]) {
    case '':
        f(s+1);
    case '\0':
        break;
    default:
        f(s+1);
        if( s[0]>='a' && s[0]<='z')
            printf("%c", s[0]);
        else if (s[0]>='A' && s[0]<='Z')
            printf("%c", s[0]-'A'+'a');
        else printf(">");
    } //switch
} //f
int main()
{
    f("Ana are Mere!");
    return 0;
}
```

'\0' ← caracterul nul ⇒ sfârșitul șirului

↳ transformă literele mari în mici
ex: 'B' - 'A' + 'a' ⇒ 66 - 65 + 97 ⇒ 98 ⇒ 'b'

⇒ >eremarama (valoarea de la sfârșit către început, din cauza apelurilor recursive)

2.2.
```
int main (void) {
    int x[2] = {5,6};
    const int *p = &x;
    printf("%d\n", *(++p));
    return 0;
}
```

CORECT ERA ⇒
```
int x[2] = {5,6};
int *p = x;
printf("%d\n", *(++p));
```
⇒ 6

↑ pointerul e incrementat și apoi dereferențiat pt a-și lua val. din noua poziție

⇒ eroare de compilare, pt că am declarat *p cu const int, ceea ce înseamnă că eu nu mai am voie să modific p-ul, dar în printf este ++p, ceea ce va rezulta la o eroare de compilare

2.3.
```
int main(){
    static int i;
    for(i++; ++i; i++){
        printf("%d", i);
        if(i==6)
            break;
    }
    return 0;
}
```

⇒ i=0

cond de continuare

++i; ~~cum e 0 la început ⇒ cond de oprire~~ ~~când i ajunge~~

adică ++i ≠ 0

~~calculul meu~~ ~~pt ++i vezi 1~~

⇒ 2 4 6

i=0

i++ ⇒ i=1

verifică cond de cont ⇒ face ++i ⇒ i=2 ≠0

intră în for și afișează (2)

face incrementarea (~~cond de continuare~~) ⇒ face i++ ⇒ i=3

i=3

verifică cond de cont ⇒ face ++i ⇒ i=4 ≠0

intră în for și afișează (4)

face incrementarea ⇒ face i++ ⇒ i=5

i=5

verifică cond de cont ⇒ face ++i ⇒ i=6 ≠0

intră în for și afișează (6)

if(i==6) ⇒ break;

• 6.02.2018

2. SUBIECTUL 2

2.1.
```
int main(){
char * cities[] = {"UAE", "Spain", "America"};
int **i = &cities[0]; => U
int **j = &cities[1]; => S
int **k = &cities[2]; => A
printf("%c%c%c\n", **i, **j, **k);
return 0;
}
```
=> USA, dar vor fi și warning-uri pt că inițializăm int** cu un pointer de tip char **

2.2.
```
int main(){
  static int i;
  for(i++; ++i; i++){
    printf("%d", i);
    if(i==6)
      break;
  }
  return 0;
}
```
=> 2 4 6 EXPLICAȚIA LA PAG ~2v (a mai fost ex asta o dată)

2.3. #define ARIA(latura) latura * latura

```
int main(void){
    int i=0;
    for(;i<3;){
        printf("%d\n", ARIA(i+1));
    }
}
```

i=0
ARIA(1) = 1 * 1 = 1
i=0 < 3
ARIA(1) = 1 * 1 = 1

} => o să afișeze 1 la infinit pt că i rămâne 0 și e mereu < 3

=> se poate ajunge la un comportament neașteptat în ordinea evaluării din cauză că MACRO-ul ARIA(latura) (nu!) are parantezele în jurul argumentelor

ex: pt printf("%d\n", ARIA(i+1)); macro-ul va fi înlocuit și va =>

=> printf("%d\n", i+1 * i+1); → ordinea evaluării va fi i+1, apoi (i+1)*i, apoi [(i+1)*i]+1

ar fi trebuit să fie ((latura) * (latura))

• 23 ianuarie 2023

2.2 2. SUBIECTUL 2

2.1.
```
void *inc(void *p, int size){
    return ((char*)p)+size;
}
int main(){
    double v[3]={1,2,3};
    double *p=inc(v, sizeof(double));
    printf("%lf\n", *p);
    return 0;
}
```

return ((char*)p)+size; ⇒ 1 + size ⇒ al size-lea byte din vectorul dacă aveam ((int*)p)+size ⇒ 4 + size

v[0] ⇒ v[1] ⇒ 2

((char*)v) + 8 octeți { trec la următorul element din vector

8 octeți

2.2 afișează un număr cu 6 zecimale implicit

ex: dacă aveam %.2lf ⇒ se va afișa un cu 2 zecimale

⇒ 2.000000

2.2.
```
int main(){
    unsigned int b=1;
    b<<=(1&2&3);
    printf("%d\n", b);
    b=1<<31;
    printf("%u %d\n", b, b);
    return 0;
}
```

1&2 : 0001 & 0010 ⇒ 0000 = 0

(1&2)&3 : 0000 & 0011 ⇒ 0000 = 0

b<<=0 ⇒ b rămâne neschimbat ⇒ b=1

1<<31 <⇒ 2^{31}

1<<3 <⇒ 2^3 0001 <<3 ⇒ 1000 ⇒ 2^3

⇒ 1
2^{31} -2^{31}

%d <⇒ signed ⇒ când primul bit e 1 ⇒ se scade primul bit

%u <⇒ unsigned ⇒ când primul bit e 1 ⇒ se adună primul bit !

2.3.
```
int main(){
    char* countries[]={"Uruguay", "Spain", "Andorra"};
    int ***i = &countries[0];
    int ***j = &countries[1];
    int ***k = &countries[2];
    printf("%c%c%c\n", ***i, ***j, ***k);
    return 0;
}
```

⇒ USA

⇒ poate avea warning din cauză că inițializez un pointer de tip int*** cu unul de tip char**

(a mai fost o dată ⇒ pag ~3~)

- 23 ianuarie 2023 ~m2~

2. SUBIECTUL 2

2.1.
```
int incOarica(void *p) {
    return (*p)+1;
}
int main(){
    int x=1;
    printf("%d\n", inc(&x));
    return 0;
}
```

← aici îl dereferențiază pe p și nu va mai fi ignorat

⇒ eroare de compilare

și warning pt că funcția inc nu există

2.2.
```
int main(){
    double x=1;
    double (*f[5])(double)={sin,cos,tan,exp,log};
    printf("%lf\n", f[((int)x)<<2](x));
    return 0;
}
```

$\Rightarrow$ 0.000000

(int)x $\Rightarrow$ 1

((int)x)<<2 $\Rightarrow$ 1<<2 $\Rightarrow$ 0001<<2 $\Rightarrow$ 0100 $\Rightarrow 2^2=4$

f[((int)x)<<2] $\Rightarrow$ f[4] $\Rightarrow$ log

f[((int)x)<<2](x) $\Rightarrow$ log x, $x=1$ $\Rightarrow \log_e 1 = 0$

%lf $\Rightarrow$ 6 zecimale $\Rightarrow$ 0.000000

2.3.
```
int main(){
    static int i;
    for(i++;++i;i++){
        printf("%d", i);
        if(i==6)
            break;
    }
    return 0;
}
```

$\Rightarrow$ 2 4 6 (explicația la pag~2v a mai fost ânta de 2 ori)

• 5.02.2018

2. SUBIECTUL 2

2.1.
```
void f(char *k){
    k++;          // hello → ello
    k[2]='m';
    printf("%s\n",k);
}
void main(void){
    char s[]="hello";
    f(s);
}
```
→ pointerul k punctează către al doilea caracter al șirului

⇒ elmo

2.2.
```
int main(void){
    char i=55;
    for(;i;i++);
    printf("%d\n",i);
    return 0;
}
```
codul ASCII pt "7"

codul ASCII pt "0" este 48

char are valori între -2^7 și 2^7-1 <=> -128 și 127

i va crește până când va ajunge 127 și apoi când o să crească iar, o va lua de la i=-128, iar for-ul va continua să ruleze până când i va fi =0, iar atunci se va opri

i!=0 cond de continuare

i = 0

⇒ 0

2.3.
```
int f(){
    static int c;
    return ++c;
}
int main(){
    printf("%d\n",f());
    printf("%d\n",f()+f());
    return 0;
}
```
c=0

primul apel f() ⇒ 1 (c=1)

al doilea apel f() ⇒ 2 (c=2)

al treilea apel f() ⇒ 3 (c=3)

→ 1

→ 2+3=5

⇒ 1
5

• 7.02.2018

2. SUBIECTUL 2

2.1.
```
#define STOP 4
int fun(int m){
    if(m==STOP)
        return m;
    else return 2*fun(m+1)+1;
}
int main(){
    printf("%d", fun(1));
    return 0;
}
```
⇒ 39

• fun(1)
m=1
→ 2*fun(2)+1 ⇒ 2·19+1 = 39
• fun(2)
→ 2*fun(3)+1 ⇒ 2·9+1 = 19
• fun(3)
→ 2*fun(4)+1 ⇒ 2·4+1 = 9
• fun(4)
⇒ 4

2.2.
```
#define N 10
void fun(int *v){
    int i, start=v[0];
    for(i=1; i<N && v[i]!=start; i++)
        printf("%d", v[i]);
}
int main(){
    int *v, i;
    v=(int*)malloc(N*sizeof(int));
    if(v){
        for(i=0; i<N; i++)
            v[i]=i;
        *v=6;
        fun(v);
    }
    return 0;
}
```
cât timp v[i]!=6

for(i=0; i<10; i++) v[i]=i;
→ v[0]=0
v[1]=1
v[9]=9
v[10] = {0,1,2,3,4,5,6,7,8,9}
*v=6 ⇒ v[10]={6,1,2,3,4,5,6,7,8,9}
fun(v);
start=v[0]=6

⇒ 1 2 3 4 5

*v=6 ← atribuie valoarea 6 primului elem al vectorului !

2.3. typedef int (*intFun)(int, int);

pointer la o funcție care primește 2 parametrii de tip int și returnează un int

```
int max(int x, int y){
    return (x>y ? x : y);
}
int main(){
    int x=5, y=7;
    intFun fun;
    fun = max;
    printf("%d %p %p\n", fun(x,y), max, fun);
    return 0;
}
```

dacă x>y => return x
dacă x<y => return y

!

fun = pointer la funcție de tipul intFun

atribuie pointerului fun adresa funcției max a.î. fun să poată fi utilizat pt a apela funcția max

%p = adresa unde e stocat ceva în memorie

fun(x,y) ⇕ max(x,y) = 7 <=> max(5,7) 7>5 => 7

adresa unde e stocată funcția max în memorie

adresa unde este stocat pointerul fun în memorie

=> 7 adresa fct max adresa pointerului fun

- 4 februarie 2016
2. SUBIECTUL 2

2.1.
```
typedef struct{
    char a;
    int b;
    char*c;
}TTest;
int main(void){
    TTest*a = malloc(10*sizeof(TTest));
    TTest b[10];
    printf("%d %d\n", sizeof(a), sizeof(b));
    return 0;
}
```

=> 8 90

10*sizeof(TTest) = 10*[sizeof(char)+sizeof(int)+sizeof(char*)] = 10*(1+4+4) = 10*9 = 90

dimensiunea unui pointer este 8 octeți

!

2.2. `int (*f1(int (*a)(int, int)))(int, int);`

// precizați doar ce face linia de cod de mai sus

a => pointer la o funcție care primește 2 parametri de tip int și returnează un int

funcția f_1 primește ca argument un pointer la o altă funcție ("a"), care are 2 param de tip int și returnează un int, și returnează un pointer la o altă funcție care primește 2 param de tip int și returnează un int.

2.3

```
int main(){
    int i, j, m[5][5];
    for(i=0; i<25; i++)
        *(*m+i) = i;
    i=0, j=0;
    for(; i<5; i++){
        for(; j<5; j++)
            printf("%d", m[i][j]);
    }
    printf("\n");
    return 0;
}
```

*(*m+i) = i; ← adresa elem de la poz i valoarea ei

declar o matrice de 5 pe 5 și o inițializează cu val de la 0 la 24
afișează prima linie pt că j e inițializat o singură dată și după afișează numai linii goale pt că j nu ajunge la 5

=> 0 1 2 3 4

7 • 8 februarie 2017

2. SUBIECTUL 2

2.1
```
int f(){
    static int c;
    return ++c;
}
int main(){
    printf("%d\n", f()); → 1
    printf("%d\n", f()+f()); → 2+3=5
    return 0;
}
```
⇒ 1
5

(explicație pag ~8N, a mai fost o dată)

2.2.
```
int main(void){
    char *a = "Ana are mere și pere";
    int *p = a;
    p += 2;
    a = p;
    printf("%s\n", a);
    return 0;
}
```
⇒ a are mere și pere + warning pt că inițializez int* cu char* (are alt tip)

!

2.3
```
int main(void){
    int x[2] = {5,6};
    const int *p = &x;
    printf("%d\n", *(++p));
    return 0;
}
```
nu am voie să îl modific dacă e cu const int

⇒ eroare de compilare

(explicație pag ~2N, a mai fost o dată)

• 8 februarie 2017

2 SUBIECTUL 2

2.1. int main(void){ ← literal constant

```
char *p1 = "abcde";
char *p = p1;
p++;
*p = 'x';
printf("%s\n", p);
return 0;
}
```

p1 → abcde
p → abcde
p → bcde

→ va fi un comportament nedefinit, deoarece șirurile literale sunt considerate constante, iar modificarea lor poate duce la comportament imprevizibil

⇒ comportament nedefinit, rezultatul depinde de implementarea compilatorului

2.2. int main(void){

```
int (*f[10])(const char *, const char *) = {strcmp, strcspn, strcoll, strspn};
char s1[20] = "am examen azi";
char s2[20] = "si iau 10";
printf("%d\n", (*f)(s1, s2));
return 0;
}
```

funcțiile sunt de tipul int, deci trb accesate cu f[i]!

f este un vector de pointeri la funcții

eroare de sintaxă ce va genera un mesaj de eroare la compilare

- ca să accesez o funcție specifică din vector, ar trebui să scriu așa ceva: f[0](s1, s2) !

⇒ eroare de sintaxă

2.3.
```
int main(void){
  int i, j, m[5][5];
  for(i=0; i<25; i++)
    *(*m+i)=i;
  i=0; j=0;
  for(; i<5; i++){
    for(; j<5; j++)
      printf("%d", m[i][j]);
    printf("\n");
  }
}
```
⇒ 0 1 2 3 4

(explicație ~pag 11~, a mai fost o dată)

• 3 februarie 2022
2. SUBIECTUL 2

2.1.
```
int f(int n, int *sum){
  int ret;
  if(n==0 || n==1)
    ret=0;
  else if(n==2)
    ret=1;
  else ret=f(n-1,sum)+f(n-2,sum)+f(n-3,sum);
  *sum+=ret;
  return ret;
}
int main(){
  int i, sum=0;
  i=f(6,&sum);
  printf("%d %d\n", i, sum);
  return 0;
}
```

→ i=f(6, &sum)
ret = f(5,sum) + f(4,sum) + f(3,sum);

→ f(5,&sum)
ret = f(4,sum) + f(3,sum) + f(2,sum);

→ f(4,&sum)
ret = f(3,sum) + f(2,sum) + f(1,sum);

→ f(3,&sum)
ret = f(2,sum) + f(1,sum) + f(0,sum);

→ f(2,&sum) ⇒ ret=1 sum=0+1=1
→ f(1,&sum) ⇒ ret=0 sum=1+0=1
→ f(0,&sum) ⇒ ret=0 sum=1+0=1
→ f(3,&sum) ⇒ ret=1+0+0=1 sum=1+1=2
→ f(4,&sum) ⇒ ret=1+1+0=2 sum=2+2=4
→ f(5,&sum) ⇒ ret=2+1+1=4 sum=4+4=8
→ f(6,&sum) ⇒ ret=4+2+1=7 sum=8+7=15

⇒ 7 15

2.2.
```
int main(){
    int i;
    for(i=0;i<5;i++){
        printf("%d ",i);
        switch(i){
            case 2: printf("2\n"); break;
            case 3: printf("3\n");
            case 4: printf("4\n"); break;
            case 5: printf("5\n"); break;
            default: printf("def\n"); break;
        }
    }
}
```

pt i=0 => 0 def
pt i=1 => 1 def
pt i=2 => 2 2
pt i=3 => 3 3 4
pt i=4 => 4 4

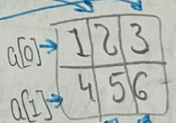
! pt i=3 când intră pe case 3, merge și îl face și pe case 4

2.3.
```
int main(){
    int a[][3]={1,2,3,4,5,6};
    int (*ptr)[3]=a;
    printf("%d %d", (*ptr)[1], (*ptr)[2]);
    ++ptr;
    printf("%d%d\n", (*ptr)[1], (*ptr)[2]);
    return 0;
}
```

// se declară și se inițializează o matrice cu 2 linii și 3 coloane

ptr = a[0]

++ptr; => a[1]



! ptr este un pointer la un vector a[3] cu 3 elemente

=> 2 3
5 6

4 februarie 2022

2. SUBIECTUL 2

2.1.

```
void fun(char *p){
    if(p==NULL || *p=='\0')
        return 0;
    int current=1, i=1;
    while(*(p+current)){
        if(p[current]!=p[current-1]){
            p[i]=p[current];
            i++;
        }
        current++;
    }
    *(p+i)='\0';
}
int main(){
    char str[]="annaaareeraannna";
    fun(str);
    puts(str);
    return 0;
}
```

afișează un șir (până la caracterul '\0') pe consolă, urmat de o linie nouă !

=> anareranna

→ elimină caracterele duplicate consecutive din șirul str

c=1, i=1

while(*(p+c)){ *(p+1)=n!='\0'

if(p[1]!=p[0]) n!=a => p[1]=p[1], i=2, c=2
*(p+2)=n!='\0'

if(p[2]!=p[1]) n=n, c=3, i=2
*(p+3)=a!='\0'

if(p[3]!=p[2]) a!=n, p[2]=p[3], i=3, c=4
anaaareeraannna
*(p+4)=a!='\0'

if(p[4]!=p[3]) a=a, i=3, c=5
*(p+5)=r!='\0'

if(p[5]!=p[4]) r!=a, p[3]=p[5], i=4, c=6
anaraeeraannna
*(p+6)=e!='\0'

if(p[6]!=p[5]) e!=r, p[4]=p[6], i=5, c=7
anareeeraannna
*(p+7)=e!='\0'

if(p[7]!=p[6]) e=e, i=5, c=8
*(p+8)=r!='\0'

if(p[8]!=p[7]) r!=e, p[5]=p[8], i=6, c=9
anareeeraannna
*(p+9)=a!='\0'

if(p[9]!=p[8]) a!=r, p[6]=p[9], i=7, c=10
anareraeraannna
*(p+10)=a!='\0'

if(p[10]!=p[9]) a=a, i=7, c=11
*(p+11)=n!='\0'

if(p[11]!=p[10]) n!=a, p[7]=p[11], i=8, c=12
anareranraannna
*(p+12)=n!='\0'

if(p[12]!=p[11]) n=n, i=8, c=13
*(p+13)=a!='\0'

if(p[13]!=p[12]) a!=n, p[8]=p[13], i=9, c=14
anareranaaannna
*(p+14)='\0'='\0'

*(p+9)='\0' => anareranа'\0'

2.2.
```
#define N 10
void foo(int *x, void *y){
    printf("%d", (char*)x-(char*)y);
    ++x;
}
int main(void){
    int i, *p;
    p = calloc(10, sizeof(int));
    void *p1 = p;
    for(i=0; i<N; i++, p++){
        foo(p, p1);
    }
    return 0;
}
```

→ afișez diferența în octeți dintre adresa curentă a lui x și adresa de început a blocului y, convertind pointerii la char* pentru a obține o diferență în octeți.

→ adun cu câte unul mai mult, adică 4 octeți

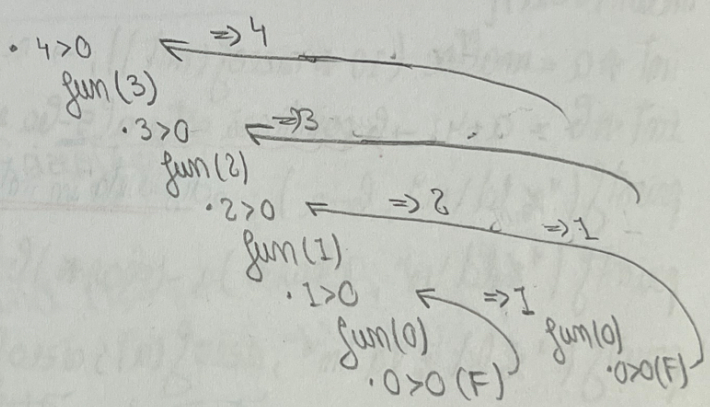
→ aloc dinamic un bloc de memorie pt 10 întregi
↘ adresa de început a acestui bloc este stocată în pointerul p

pointer la void

⇒ 0 4 8 12 16 20 24 28 32 36

2.3.
```
void fun(int m){
    if(m>0){
        fun(m-1);
        printf("%d", m);
        fun(m-1);
    }
}
int main(){
    fun(4);
    return 0;
}
```

fun(1) 1
fun(2) 2 1
fun(3) 3 1 2 1
fun(4) 4 1 2 1 3 1 2 1



⇒ 1 2 1 3 1 2 1 4 1 2 1 3 1 2 1

• 15 februarie 2022
3. SUBIECTUL 3

3.1.
```
int main(){
    int x = 512;
    void *p = &x;
    printf("%d\n", *((char*)++p));
    return 0;
}
```

p pointează la adresa variabilei x

• după ++p, p pointează la următorul octet în memorie, adică la octet-ul din reprezentarea internă a var. x
• acest octet are valoarea 0000 0010 în reprezentare binară, adică valoarea 2 în zecimal

!

=> 2

3.2.
```
int main(void){
    int *a = malloc(10 * sizeof(int));
    int *b = a+4;
    printf("%ld\n", b-a);
    printf("%ld\n", (char*)a - (char*)b);
    printf("%ld %ld\n", sizeof(a), sizeof(b));
    free(b);
    return 0;
}
```

a → realocă dinamic un bloc de memorie pt 10 întregi
b pointează către al 5-lea elem. al blocului alocat dinamic
b-a → diferența în întregi dintre pointerii b și a (adică dintre adresa b-a <=> 5-1 => 4)
diferența în octeți dintre pointerii a și b
a pointează către primul elem
b pointează către al 5-lea
a-b => 1-5 => -4
în octeți => -4 * sizeof(int) = -4*4 = -16
sizeof(a), sizeof(b) → dimensiunea unui pointer = 8 octeți (8)

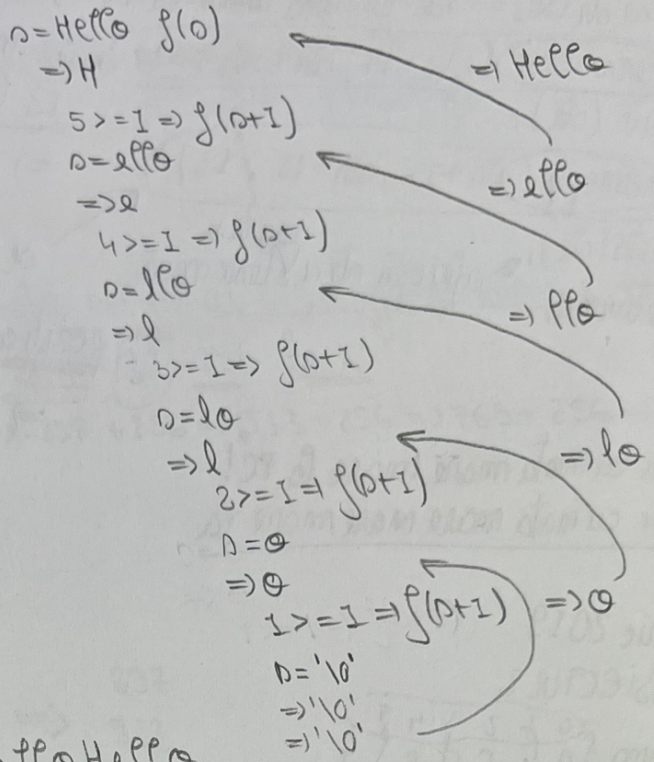
=>
4
-16
8 8

comportament nedefinit
(trebuie să fie free(a);)

• 31 ianuarie 2020

2. SUBIECTUL 2

2.1.
```
void f(char *p){
    printf("%c", *p);
    if(strlen(p)>=1){
        f(p+1);
    }
    printf("%p", p);
}
int main(){
    f("Hello");
    return 0;
}
```



=> H e ll o '\0' '\0' o l o ll o e ll o Hello

2.2.
```
int main(){
    char *x=(char*)calloc(10,sizeof(int));
    for(int i=0;i<10;++i){
        x[i]=(i%4==1);
    }
    for(int i=0;i<10;++i){
        printf("%d ", ((int*)x)[i]);
    }
    return 0;
}
```

cand i%4==1 => x[i]=1
i%4==0 => x[i]=0 !

x[0] x[1] x[2] x[3] x[4] x[5] x[6] x[7] x[8] x[9]

=> 0 1 0 0 0 1 0 0 0 1

2.3. int main(){
char str[30] = "Trec cu nota mare la PC!";
memmove(str+17, str+12, 13); ← se muta 13 caractere incepand cu pozitia 12 la pozitia 17 in sirul str
⇒ Trec cu nota mare mare la PC!"
puts(str);
memcpy(str+17, str+12, 13); ← se copiază 13 caractere incepand cu pozitia 12 la pozitia 17 in sirul str
⇒ Trec cu nota mare mare mare la PC!"
puts(str); ← afișează str și trece la o linie nouă
return 0;
}

⇒ Trec cu nota mare mare la PC!
Trec cu nota mare mare mare la

• 5 februarie 2019
SUBIECTUL 2

2.3. 0 1 2 3 4 5
enum {a, b, c, d, e, f};
struct foo {
unsigned char ch : 3;
unsigned int myint : 4; ← in reprezentarea, acest câmp poate să aibă maxim 4 biți
};
int main(void) {
struct foo f;
f.ch = b ^ c; ⇔ 1 ^ 2 (XOR) 1 ⇒ 01, 2 ⇒ 10, 1^2 ⇒ 11 ⇒ 3
printf("%d\n", f.ch); ⇒ 3
f.ch = (1 << f.ch) - f.ch; (1<<3) - 3 ⇔ 2^3 ← 3 ⇔ 8 - 3 ⇒ 5
f.myint = ~f.ch; COMPLEMENTUL LA 1 AL VALORII LUI f.ch; f.ch = 0101 ⇒ ~f.ch = 1010 (5 / 10)
printf("%d\n", f.ch); ⇒ 5
printf("%d\n", f.myint); ⇒ 10 (0 devine 1 și 1 devine 0)
return 0;
}

⇒ 3
5
10

• 5 februarie 2019

2. SUBIECTUL 2

2.1.
```
#define N 4
int fun(void *p){
  char *pc = (char*)p;
  (*(pc+1))++;
}
int main(){
  int x = 257, i=0;
  while(i<N){
    printf("%d\n", x);
    fun(&x);
    i++;
  }
  return 0;
}
```

x = 257 0000001 0000001

(*(pc+1))++ ⇔ 0000010 0000001

al 2-lea octet — primul octet

↖ incrementez cel de-al 2-lea octet al valorii x cu 1

⇔ adun $2^8 = 256$ la x

x = 257 + 256 ⇒ 513 + 256 ⇒ 769 + 256 ⇒ 1025

⇒ 257
513
769
1025

2.3.
```
int fun(int n, int *fp){
  int t, f;
  if(n<=1){
    *fp = 1;
    return 1;
  }
  t = fun(n-1, fp);
  f = t + *fp;
  *fp = t;
  return f;
}
int main(){
  int x = 15;
  printf("%d\n", fun(6, &x));
  printf("%d\n", x);
  return 0;
}
```

nu mai am chef, sorry

⇒ 13
8

• 21.01.2018
2. SUBIECTUL 2

2.1.
```
int main(void){
    int d=30;
    int *p=&d;                  -> p=30
    printf("%d %d\n", d, *p);   => 30 30
    *p=32;                      <- se modifică val la care pointează p
    printf("%d %d\n", d, *p);   => 32 32
    /* Presupunem că malloc-ul nu
    execută */
    p=malloc(sizeof(int));      <- se alocă memorie pt un int
    d=*p;                          memoria alocată nu este inițializată explicit => valoare
    p=&d;                          nenumerată
    printf("%d %d\n", d, *p);   => val.nec val.nec
    free(p);
    return 0;                   =>  30 30
}                                   32 32
                                    val.nec. val.nec.
```

atoare a variabilei d
decarece p pointează la adresa lui d.
=> d=32, *p=32

• 4 septembrie 2018
2. SUBIECTUL 2

2.1.
```
int main(void){
    struct test{
        int v[5];
    };
    int i;
    struct test a={{0,1,2,3,4}}, b={{0}};
    b=a;
    b.v[4]=100;                 => a={{0,1,2,3,4}}, b={{0,1,2,3,100}}
    for(i=0; i<5; i++)                                  0 0
        printf("%d %d\n", a.v[i], b.v[i]);      =>      1 1
    return 0;                                           2 2
}                                                       3 3
                                                        4 100
```

2.3. int main(void){
    char *str = "hello";
    char *ptr = str;
    char least = 127;
    while (*ptr++)
        least = ((*ptr)<(least)) ? (*ptr) : (least);
    printf("%d\n", least);
}

⇒ afișează valoarea minimă a caracterului ASCII găsit în șir

⇒ 0 (pt că least va fi comparat și cu terminatorul de șir, a cărui valoare în ASCII este 0)